

Сетевой клиент

1.0

Generated by Doxygen 1.9.1

1 Class Index	1
1.1 Class List.....	1
2 File Index	3
2.1 File List.....	3
3 Class Documentation	5
3.1 AuthManager Class Reference.....	5
3.1.1 Detailed Description	5
3.1.2 Constructor & Destructor Documentation.....	5
3.1.2.1 AuthManager().....	5
3.1.2.2 ~AuthManager().....	6
3.1.3 Member Function Documentation	6
3.1.3.1 computeHash()	6
3.1.3.2 generateSalt()	6
3.2 ClientParams Struct Reference.....	6
3.2.1 Detailed Description	7
3.2.2 Member Data Documentation	7
3.2.2.1 config_file.....	7
3.2.2.2 input_file	7
3.2.2.3 output_file	7
3.2.2.4 port	8
3.2.2.5 server_address	8
3.3 ConnectionManager Class Reference.....	8
3.3.1 Detailed Description	9
3.3.2 Constructor & Destructor Documentation.....	9
3.3.2.1 ConnectionManager()	9
3.3.2.2 ~ConnectionManager()	9
3.3.3 Member Function Documentation	9
3.3.3.1 authenticate()	9
3.3.3.2 processVectors().....	10
3.3.3.3 receiveData()	10
3.3.3.4 receiveString().....	10
3.3.3.5 sendData()	10
3.3.3.6 sendString().....	11
3.3.4 Member Data Documentation	11
3.3.4.1 port_.....	11
3.3.4.2 server_addr_	11
3.3.4.3 socket_.....	11
3.4 FileHandler Class Reference.....	12
3.4.1 Detailed Description	12
3.4.2 Constructor & Destructor Documentation.....	12
3.4.2.1 FileHandler() [1/2]	12
3.4.2.2 ~FileHandler().....	12

3.4.2.3 FileHandler() [2/2]	12
3.4.3 Member Function Documentation	13
3.4.3.1 operator=()	13
3.4.3.2 readConfig()	13
3.4.3.3 readVectors()	13
3.4.3.4 saveResults()	13
3.5 UserInterface Class Reference	14
3.5.1 Detailed Description	14
3.5.2 Constructor & Destructor Documentation	14
3.5.2.1 UserInterface()	14
3.5.3 Member Function Documentation	15
3.5.3.1 getHelp()	15
3.5.3.2 getParams()	15
3.5.3.3 parseArguments()	15
3.5.4 Member Data Documentation	15
3.5.4.1 desc_	15
3.5.4.2 params_	15
4 File Documentation	17
4.1 src/auth_manager.cpp File Reference	17
4.1.1 Detailed Description	17
4.2 src/auth_manager.h File Reference	17
4.2.1 Detailed Description	18
4.2.2 Macro Definition Documentation	18
4.2.2.1 CRYPTOPP_ENABLE_NAMESPACE_WEAK	19
4.3 src/connection_manager.cpp File Reference	19
4.3.1 Detailed Description	19
4.4 src/connection_manager.h File Reference	19
4.4.1 Detailed Description	20
4.5 src/file_handler.cpp File Reference	20
4.5.1 Detailed Description	20
4.6 src/file_handler.h File Reference	21
4.6.1 Detailed Description	21
4.7 src/main.cpp File Reference	22
4.7.1 Detailed Description	22
4.7.2 Function Documentation	22
4.7.2.1 main()	22
4.8 src/user_interface.cpp File Reference	23
4.8.1 Detailed Description	23
4.9 src/user_interface.h File Reference	23
4.9.1 Detailed Description	24
Предметный указатель	25

Глава 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AuthManager	Статический класс для криптографических операций.....	5
ClientParams	Структура для хранения параметров клиента	6
ConnectionManager	Класс для управления TCP-соединением с сервером	8
FileHandler	Статический класс для операций с файлами	12
UserInterface	Класс для обработки параметров командной строки.....	14

Глава 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

src/auth_manager.cpp	Реализация класса аутентификации.....	17
src/auth_manager.h	Заголовочный файл класса аутентификации	17
src/connection_manager.cpp	Реализация класса управления соединением	19
src/connection_manager.h	Заголовочный файл класса управления соединением	19
src/file_handler.cpp	Реализация класса работы с файлами	20
src/file_handler.h	Заголовочный файл класса работы с файлами	21
src/main.cpp	Главный модуль клиентского приложения	22
src/user_interface.cpp	Реализация класса пользовательского интерфейса	23
src/user_interface.h	Заголовочный файл класса пользовательского интерфейса	23

Глава 3

Class Documentation

3.1 AuthManager Class Reference

Статический класс для криптографических операций

```
#include <auth_manager.h>
```

Static Public Member Functions

- static std::string [generateSalt](#) ()
- static std::string [computeHash](#) (const std::string &salt, const std::string &password)

Private Member Functions

- [AuthManager](#) ()=delete
- [~AuthManager](#) ()=delete

3.1.1 Detailed Description

Статический класс для криптографических операций

Definition at line 15 of file auth_manager.h.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 AuthManager()

AuthManager::AuthManager () [private], [delete]

3.1.2.2 ~AuthManager()

AuthManager::~AuthManager () [private], [delete]

3.1.3 Member Function Documentation

3.1.3.1 computeHash()

```
std::string AuthManager::computeHash (
    const std::string & salt,
    const std::string & password ) [static]
```

Definition at line 26 of file auth_manager.cpp.

Referenced by ConnectionManager::authenticate().

3.1.3.2 generateSalt()

```
std::string AuthManager::generateSalt ( ) [static]
```

Definition at line 16 of file auth_manager.cpp.

Referenced by ConnectionManager::authenticate().

The documentation for this class was generated from the following files:

- [src/auth_manager.h](#)
- [src/auth_manager.cpp](#)

3.2 ClientParams Struct Reference

Структура для хранения параметров клиента

```
#include <user_interface.h>
```

Public Attributes

- std::string [server_address](#)
Адрес сервера
- uint16_t [port](#) = 33333
Порт сервера
- std::string [input_file](#)
Входной файл
- std::string [output_file](#)
Выходной файл
- std::string [config_file](#) = "~/config/vclient.conf"
Файл конфигурации

3.2.1 Detailed Description

Структура для хранения параметров клиента

Definition at line 14 of file user_interface.h.

3.2.2 Member Data Documentation

3.2.2.1 config_file

```
std::string ClientParams::config_file = "~/config/vclient.conf"
```

Файл конфигурации

Definition at line 19 of file user_interface.h.

Referenced by `UserInterface::UserInterface()`.

3.2.2.2 input_file

```
std::string ClientParams::input_file
```

Входной файл

Definition at line 17 of file user_interface.h.

Referenced by `UserInterface::UserInterface()`.

3.2.2.3 output_file

```
std::string ClientParams::output_file
```

Выходной файл

Definition at line 18 of file user_interface.h.

Referenced by `UserInterface::UserInterface()`.

3.2.2.4 port

```
uint16_t ClientParams::port = 33333
```

Порт сервера

Definition at line 16 of file `user_interface.h`.

Referenced by `ManagerInterface::ManagerInterface()`.

3.2.2.5 server_address

```
std::string ClientParams::server_address
```

Адрес сервера

Definition at line 15 of file `user_interface.h`.

Referenced by `main()`, and `ManagerInterface::ManagerInterface()`.

The documentation for this struct was generated from the following file:

- [src/user_interface.h](#)

3.3 ConnectionManager Class Reference

Класс для управления TCP-соединением с сервером

```
#include <connection_manager.h>
```

Public Member Functions

- [ConnectionManager](#) (const std::string &server_addr, uint16_t port)
- virtual [~ConnectionManager](#) ()
- void [authenticate](#) (const std::string &login, const std::string &password)
- std::vector< int32_t > [processVectors](#) (const std::vector< std::vector< int32_t > > &vectors)

Protected Member Functions

- virtual void [sendData](#) (const void *data, size_t size)
- virtual void [receiveData](#) (void *data, size_t size)
- virtual void [sendString](#) (const std::string &str)
- virtual std::string [receiveString](#) (size_t max_length=1024)

Protected Attributes

- int [socket_](#)
- std::string [server_addr_](#)
- uint16_t [port_](#)

3.3.1 Detailed Description

Класс для управления TCP-соединением с сервером

Definition at line 21 of file connection_manager.h.

3.3.2 Constructor & Destructor Documentation

3.3.2.1 ConnectionManager()

```
ConnectionManager::ConnectionManager (
    const std::string & server_addr,
    uint16_t port )
```

Definition at line 12 of file connection_manager.cpp.

References port_, server_addr_, and socket_.

3.3.2.2 ~ConnectionManager()

```
ConnectionManager::~ConnectionManager ( ) [virtual]
```

Definition at line 49 of file connection_manager.cpp.

References socket_.

3.3.3 Member Function Documentation

3.3.3.1 authenticate()

```
void ConnectionManager::authenticate (
    const std::string & login,
    const std::string & password )
```

Definition at line 55 of file connection_manager.cpp.

References AuthManager::computeHash(), AuthManager::generateSalt(), receiveString(), and sendString().

Referenced by main().

3.3.3.2 processVectors()

```
std::vector< int32_t > ConnectionManager::processVectors (
    const std::vector< std::vector< int32_t >> & vectors )
```

Definition at line 73 of file connection_manager.cpp.

References `receiveData()`, and `sendData()`.

Referenced by `main()`.

3.3.3.3 receiveData()

```
void ConnectionManager::receiveData (
    void * data,
    size_t size ) [protected], [virtual]
```

Definition at line 111 of file connection_manager.cpp.

References `socket_`.

Referenced by `processVectors()`.

3.3.3.4 receiveString()

```
std::string ConnectionManager::receiveString (
    size_t max_length = 1024 ) [protected], [virtual]
```

Definition at line 131 of file connection_manager.cpp.

References `socket_`.

Referenced by `authenticate()`.

3.3.3.5 sendData()

```
void ConnectionManager::sendData (
    const void * data,
    size_t size ) [protected], [virtual]
```

Definition at line 98 of file connection_manager.cpp.

References `socket_`.

Referenced by `processVectors()`, and `sendString()`.

3.3.3.6 sendString()

```
void ConnectionManager::sendString (
    const std::string & str ) [protected], [virtual]
```

Definition at line 127 of file `connection_manager.cpp`.

References `sendData()`.

Referenced by `authenticate()`.

3.3.4 Member Data Documentation

3.3.4.1 port_

```
uint16_t ConnectionManager::port_ [protected]
```

Definition at line 32 of file `connection_manager.h`.

Referenced by `ConnectionManager()`.

3.3.4.2 server_addr_

```
std::string ConnectionManager::server_addr_ [protected]
```

Definition at line 31 of file `connection_manager.h`.

Referenced by `ConnectionManager()`.

3.3.4.3 socket_

```
int ConnectionManager::socket_ [protected]
```

Definition at line 30 of file `connection_manager.h`.

Referenced by `ConnectionManager()`, `receiveData()`, `receiveString()`, `sendData()`, and `~ConnectionManager()`.

The documentation for this class was generated from the following files:

- [src/connection_manager.h](#)
- [src/connection_manager.cpp](#)

3.4 FileHandler Class Reference

Статический класс для операций с файлами

```
#include <file_handler.h>
```

Static Public Member Functions

- static std::vector< std::vector< int32_t > > [readVectors](#) (const std::string &filename)
- static void [saveResults](#) (const std::string &filename, const std::vector< int32_t > &results)
- static std::pair< std::string, std::string > [readConfig](#) (const std::string &filename)

Private Member Functions

- [FileHandler](#) ()=delete
- [~FileHandler](#) ()=delete
- [FileHandler](#) (const [FileHandler](#) &)=delete
- [FileHandler](#) & [operator=](#) (const [FileHandler](#) &)=delete

3.4.1 Detailed Description

Статический класс для операций с файлами

Definition at line 18 of file file_handler.h.

3.4.2 Constructor & Destructor Documentation

3.4.2.1 FileHandler() [1/2]

`FileHandler::FileHandler ( )` [private], [delete]

3.4.2.2 ~FileHandler()

`FileHandler::~~FileHandler ( )` [private], [delete]

3.4.2.3 FileHandler() [2/2]

`FileHandler::FileHandler (`
 const [FileHandler](#) &) [private], [delete]

3.13.3 Member Function Documentation

3.4.3.1 operator=()

```
FileHandler& FileHandler::operator= (
    const FileHandler & ) [private], [delete]
```

3.4.3.2 readConfig()

```
std::pair< std::string, std::string > FileHandler::readConfig (
    const std::string & filename ) [static]
```

Definition at line 70 of file file_handler.cpp.

Referenced by main().

3.4.3.3 readVectors()

```
std::vector< std::vector< int32_t > > FileHandler::readVectors (
    const std::string & filename ) [static]
```

Definition at line 11 of file file_handler.cpp.

Referenced by main().

3.4.3.4 saveResults()

```
void FileHandler::saveResults (
    const std::string & filename,
    const std::vector< int32_t > & results ) [static]
```

Definition at line 50 of file file_handler.cpp.

Referenced by main().

The documentation for this class was generated from the following files:

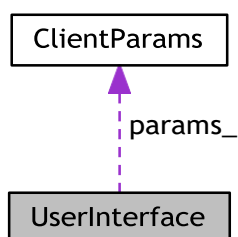
- [src/file_handler.h](#)
- [src/file_handler.cpp](#)

3.5 UserInterface Class Reference

Класс для обработки параметров командной строки

```
#include <user_interface.h>
```

Collaboration diagram for UserInterface:



Public Member Functions

- [UserInterface](#) ()
- bool [parseArguments](#) (int argc, char **argv)
- [ClientParams](#) [getParams](#) () const
- std::string [getHelp](#) () const

Private Attributes

- boost::program_options::options_description [desc_](#)
- [ClientParams](#) [params_](#)

3.5.1 Detailed Description

Класс для обработки параметров командной строки

Definition at line 26 of file user_interface.h.

3.5.2 Constructor & Destructor Documentation

3.5.2.1 UserInterface()

UserInterface::UserInterface ()

Definition at line 10 of file user_interface.cpp.

References [ClientParams::config_file](#), [desc_](#), [ClientParams::input_file](#), [ClientParams::output_file](#), [params_](#), [ClientParams::port](#), and [ClientParams::server_address](#).

3.15.3 Member Function Documentation

3.5.3.1 getHelp()

std::string UserInterface::getHelp () const

Definition at line 55 of file user_interface.cpp.

References desc_.

Referenced by main().

3.5.3.2 getParams()

[ClientParams](#) UserInterface::getParams () const [inline]

Definition at line 30 of file user_interface.h.

References params_.

Referenced by main().

3.5.3.3 parseArguments()

bool UserInterface::parseArguments (
int argc,
char ** argv)

Definition at line 20 of file user_interface.cpp.

References desc_.

Referenced by main().

3.5.4 Member Data Documentation

3.5.4.1 desc_

boost::program_options::options_description UserInterface::desc_ [private]

Definition at line 34 of file user_interface.h.

Referenced by getHelp(), parseArguments(), and UserInterface().

3.5.4.2 params_

[ClientParams](#) UserInterface::params_ [private]

Definition at line 35 of file user_interface.h.

Referenced by getParams(), and UserInterface().

The documentation for this class was generated from the following files:

- [src/user_interface.h](#)
- [src/user_interface.cpp](#)

Глава 4

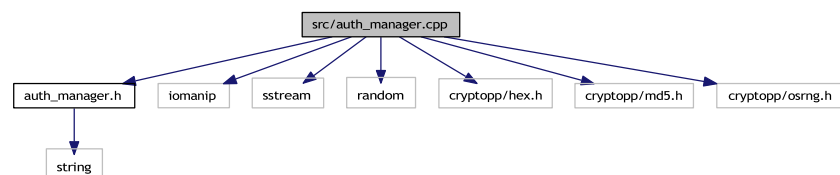
File Documentation

4.1 src/auth_manager.cpp File Reference

Реализация класса аутентификации

```
#include "auth_manager.h"  
#include <iomanip>  
#include <sstream>  
#include <random>  
#include <cryptopp/hex.h>  
#include <cryptopp/md5.h>  
#include <cryptopp/osrng.h>
```

Include dependency graph for auth_manager.cpp:



4.1.1 Detailed Description

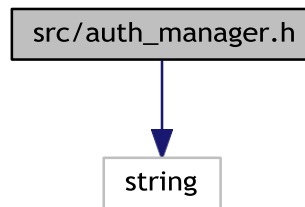
Реализация класса аутентификации

4.2 src/auth_manager.h File Reference

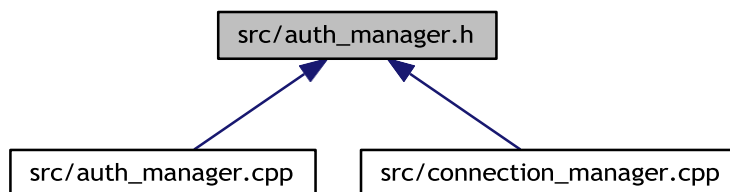
Заголовочный файл класса аутентификации

```
#include <string>
```

Include dependency graph for auth_manager.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [AuthManager](#)
Статический класс для криптографических операций

Macros

- `#define` [CRYPTOPP_ENABLE_NAMESPACE_WEAK](#) 1

4.2.1 Detailed Description

Заголовочный файл класса аутентификации

4.2.2 Macro Definition Documentation

4.2.2.1 CRYPTOPP_ENABLE_NAMESPACE_WEAK

```
#define CRYPTOPP_ENABLE_NAMESPACE_WEAK 1
```

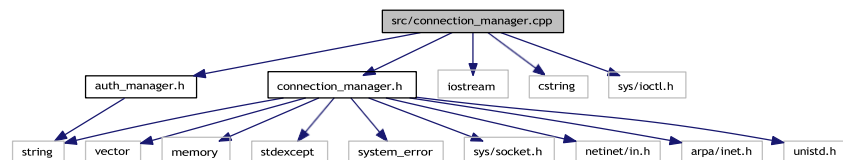
Definition at line 9 of file auth_manager.h.

4.3 src/connection_manager.cpp File Reference

Реализация класса управления соединением

```
#include "connection_manager.h"
#include "auth_manager.h"
#include <iostream>
#include <cstring>
#include <sys/ioctl.h>
```

Include dependency graph for connection_manager.cpp:



4.3.1 Detailed Description

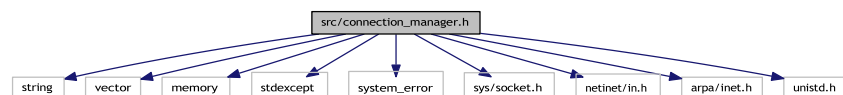
Реализация класса управления соединением

4.4 src/connection_manager.h File Reference

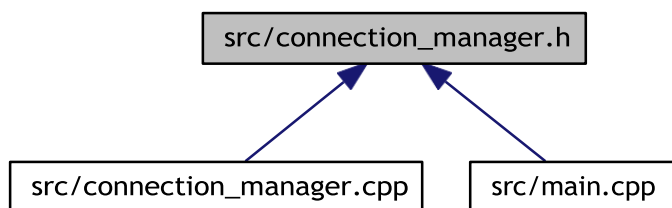
Заголовочный файл класса управления соединением

```
#include <string>
#include <vector>
#include <memory>
#include <stdexcept>
#include <system_error>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
```

Include dependency graph for connection_manager.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [ConnectionManager](#)
Класс для управления TCP-соединением с сервером

4.4.1 Detailed Description

Заголовочный файл класса управления соединением

4.5 src/file_handler.cpp File Reference

Реализация класса работы с файлами

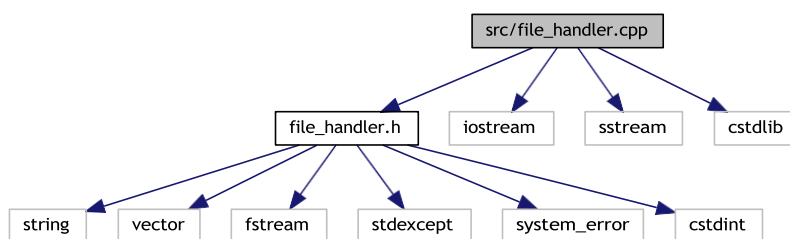
```
#include "file_handler.h"
```

```
#include <iostream>
```

```
#include <sstream>
```

```
#include <cstdlib>
```

Include dependency graph for `file_handler.cpp`:



4.5.1 Detailed Description

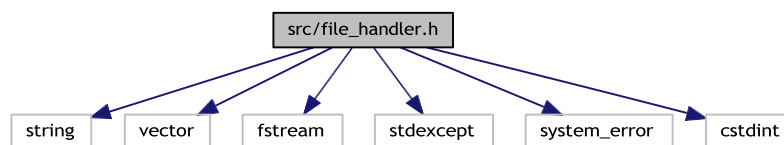
Реализация класса работы с файлами

4.6 src/file_handler.h File Reference

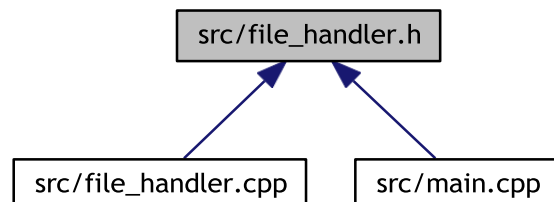
Заголовочный файл класса работы с файлами

```
#include <string>
#include <vector>
#include <fstream>
#include <stdexcept>
#include <system_error>
#include <cstdint>
```

Include dependency graph for file_handler.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [FileHandler](#)
Статический класс для операций с файлами

4.6.1 Detailed Description

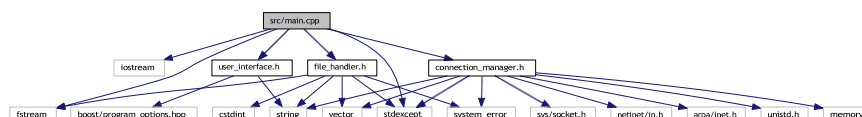
Заголовочный файл класса работы с файлами

4.7 src/main.cpp File Reference

Главный модуль клиентского приложения

```
#include <iostream>
#include <stdexcept>
#include "user_interface.h"
#include "file_handler.h"
#include "connection_manager.h"
#include <fstream>
```

Include dependency graph for main.cpp:



Functions

- `int main (int argc, char **argv)`
Точка входа в программу

4.7.1 Detailed Description

Главный модуль клиентского приложения

4.7.2 Function Documentation

4.7.2.1 main()

```
int main (
    int argc,
    char ** argv )
```

Точка входа в программу

Parameters

argc	Количество аргументов
argv	Массив аргументов

Returns

0 при успехе, 1 при ошибке

Definition at line 19 of file main.cpp.

References `ConnectionManager::authenticate()`, `UserInterface::getHelp()`, `UserInterface::getParams()`, `UserInterface::parseArguments()`, `ConnectionManager::processVectors()`, `FileHandler::readConfig()`, `FileHandler::readVectors()`, `FileHandler::saveResults()`, and `ClientParams::server_address`.

4.8 src/user_interface.cpp File Reference

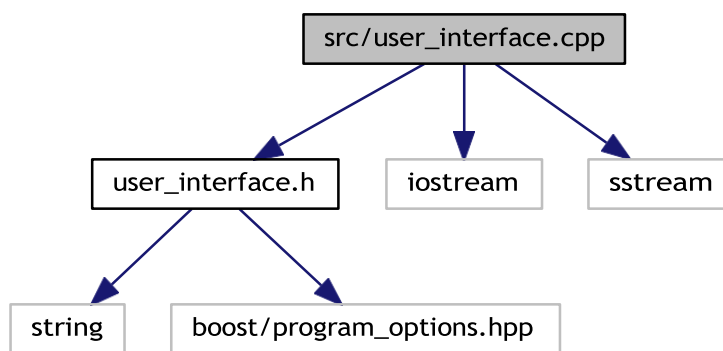
Реализация класса пользовательского интерфейса

```
#include "user_interface.h"
```

```
#include <iostream>
```

```
#include <sstream>
```

Include dependency graph for user_interface.cpp:



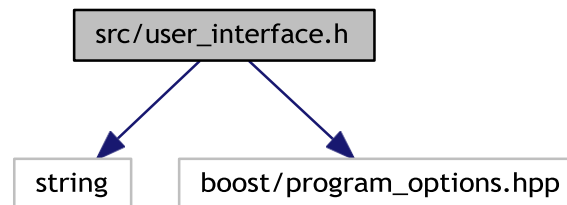
4.8.1 Detailed Description

Реализация класса пользовательского интерфейса

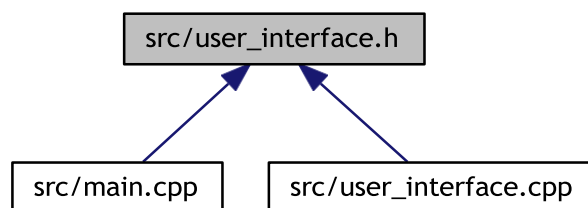
4.9 src/user_interface.h File Reference

Заголовочный файл класса пользовательского интерфейса

```
#include <string>
#include <boost/program_options.hpp>
Include dependency graph for user_interface.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- struct [ClientParams](#)
Структура для хранения параметров клиента
- class [UserInterface](#)
Класс для обработки параметров командной строки

4.9.1 Detailed Description

Заголовочный файл класса пользовательского интерфейса

Предметный указатель

- ~AuthManager
 - AuthManager, [5](#)
- ~ConnectionManager
 - ConnectionManager, [9](#)
- ~FileHandler
 - FileHandler, [12](#)
- auth_manager.h
 - CRYPTOPP_ENABLE_NAMESPACE_WEAK, [18](#)
- authenticate
 - ConnectionManager, [9](#)
- AuthManager, [5](#)
 - ~AuthManager, [5](#)
 - AuthManager, [5](#)
 - computeHash, [6](#)
 - generateSalt, [6](#)
- ClientParams, [6](#)
 - config_file, [7](#)
 - input_file, [7](#)
 - output_file, [7](#)
 - port, [7](#)
 - server_address, [8](#)
- computeHash
 - AuthManager, [6](#)
- config_file
 - ClientParams, [7](#)
- ConnectionManager, [8](#)
 - ~ConnectionManager, [9](#)
 - authenticate, [9](#)
 - ConnectionManager, [9](#)
 - port_, [11](#)
 - processVectors, [9](#)
 - receiveData, [10](#)
 - receiveString, [10](#)
 - sendData, [10](#)
 - sendString, [10](#)
 - server_addr_, [11](#)
 - socket_, [11](#)
- CRYPTOPP_ENABLE_NAMESPACE_WEAK
 - auth_manager.h, [18](#)
- desc_
 - UserInterface, [15](#)
- FileHandler, [12](#)
 - ~FileHandler, [12](#)
 - FileHandler, [12](#)
 - operator=, [13](#)
- readConfig, [13](#)
- readVectors, [13](#)
- saveResults, [13](#)
- generateSalt
 - AuthManager, [6](#)
- getHelp
 - UserInterface, [15](#)
- getParams
 - UserInterface, [15](#)
- input_file
 - ClientParams, [7](#)
- main
 - main.cpp, [22](#)
- main.cpp
 - main, [22](#)
- operator=
 - FileHandler, [13](#)
- output_file
 - ClientParams, [7](#)
- params_
 - UserInterface, [15](#)
- parseArguments
 - UserInterface, [15](#)
- port
 - ClientParams, [7](#)
- port_
 - ConnectionManager, [11](#)
- processVectors
 - ConnectionManager, [9](#)
- readConfig
 - FileHandler, [13](#)
- readVectors
 - FileHandler, [13](#)
- receiveData
 - ConnectionManager, [10](#)
- receiveString
 - ConnectionManager, [10](#)
- saveResults
 - FileHandler, [13](#)
- sendData
 - ConnectionManager, [10](#)
- sendString
 - ConnectionManager, [10](#)

- server_addr_
 - ConnectionManager, [11](#) server_address
 - ClientParams, [8](#) socket_
 - ConnectionManager, [11](#)
- src/auth_manager.cpp, [17](#)
- src/auth_manager.h, [17](#)
- src/connection_manager.cpp, [19](#)
- src/connection_manager.h, [19](#)
- src/file_handler.cpp, [20](#)
- src/file_handler.h, [21](#)
- src/main.cpp, [22](#)
- src/user_interface.cpp, [23](#)
- src/user_interface.h, [23](#)

- UserInterface, [14](#)
 - desc_, [15](#)
 - getHelp, [15](#)
 - getParams, [15](#)
 - params_, [15](#)
 - parseArguments, [15](#)
 - UserInterface, [14](#)

Таблица 1 - Тестирование пользовательского интерфейса

Название теста	Тип	Входные данные	Ожидаемый результат	Полученный результат
Показать справку (короткий вариант)	Позитивный	<code>./client -h</code>	Вывод справки о параметрах программы	Выведена справка со списком параметров
Показать справку (длинный вариант)	Позитивный	<code>./client --help</code>	Вывод справки о параметрах программы	Выведена справка со списком параметров
Запуск без параметров	Позитивный	<code>./client</code>	Вывод справки (вариант использования по умолчанию)	Выведена справка
Корректные параметры	Позитивный	<code>./client -s 127.0.0.1 -i input.txt -o output.bin</code>	Запуск программы с чтением конфигурации по умолчанию	Программа запущена, ищет сервер
Все параметры	Позитивный	<code>./client -s localhost -p 33333 -i test.txt -o res.bin -c config.conf</code>	Запуск программы с указанием всех параметров	Программа запущена с указанными параметрами
Отсутствие обязательного параметра (-s)	Негативный	<code>./client -i input.txt -o output.bin</code>	Сообщение об ошибке: отсутствует обязательный параметр	Выведено: "Ошибка разбора параметров: Отсутствуют обязательные параметры"
Неизвестный параметр	Негативный	<code>./client -s 127.0.0.1 -i input.txt -o output.bin -x</code>	Сообщение об ошибке: неизвестный параметр	Выведено: "Ошибка разбора параметров: неизвестная опция -x"
Неверный формат порта	Негативный	<code>./client -s 127.0.0.1 -p abc -i input.txt -o output.bin</code>	Сообщение об ошибке: неверное значение порта	Выведено: "Ошибка разбора параметров: значение abc недопустимо для порта"

Таблица 2 - Тестирование работы с файлами

Название теста	Тип	Входные данные	Ожидаемый результат	Полученный результат
Чтение корректного файла векторов	Позитивный	<code>input.txt</code> с 3 векторами	Успешное чтение 3 векторов	Прочитано 3 вектора, размеры: [4, 3, 2]
Чтение несуществующего файла	Негативный	<code>nonexistent.txt</code>	Исключение: файл не найден	Выброшено <code>std::system_error</code> : "Не удалось открыть файл"
Чтение пустого файла	Негативный	Пустой файл	Исключение: ошибка чтения количества векторов	Выброшено <code>std::runtime_error</code> : "Ошибка чтения количества векторов"
Чтение файла с неверным форматом	Негативный	Файл с текстом вместо чисел	Исключение: ошибка парсинга чисел	Выброшено <code>std::runtime_error</code> : "Ошибка чтения значения вектора"
Сохранение результатов	Позитивный	Вектор результатов [42, 123, -456]	Создание файла <code>output.bin</code> размером 16 байт	Файл создан, размер 16 байт, данные соответствуют
Чтение конфигурации	Позитивный	Файл <code>config.conf</code> с содержимым <code>user:P@ssW0rd</code>	Получение пары (<code>login="user"</code> , <code>password="P@ssW0rd"</code>)	Пара успешно прочитана
Конфигурация с тильдой	Позитивный	<code>~/.config/vclient.conf</code>	Раскрытие пути до домашней директории	Файл найден в <code>/home/user/.config/vclient.conf</code>
Неверный формат конфигурации	Негативный	Файл без двоеточия	Исключение: неверный формат файла	Выброшено <code>std::runtime_error</code> : "Неверный формат файла конфигурации"

Таблица 3 - Тестирование аутентификации

Название теста	Тип	Входные данные	Ожидаемый результат	Полученный результат
Генерация соли	Позитивный	-	Строка из 16 шестнадцатеричных символов	Получена строка длиной 16 символов, например "4F9C429F5C6884DB"
Формат соли	Позитивный	-	Все символы в верхнем регистре, только 0-9,A-F	Все символы соответствуют hex-формату
Уникальность соли	Позитивный	3 последовательных вызова	Все три соли разные	Соли различаются
Вычисление MD5-хеша	Позитивный	Соль="0000000000000000", Пароль="test"	Хеш длиной 32 символа	Получен хеш: "D8E8FCA2DC0F896FD7CB4CB0031BA249"
Консистентность хеша	Позитивный	Одни и те же соль и пароль	Одинаковый хеш при многократном вычислении	Хеши идентичны
Формат аутентификационного сообщения	Позитивный	Логин="user", Соль="1234567890ABCDEF", Пароль="pass"	Сообщение: "user1234567890ABCDEF<MD5-хеш>"	Сформировано сообщение длиной 16+32+len(login)

Таблица 4 - Тестирование сетевого взаимодействия

Название теста	Тип	Предусловия	Входные данные	Ожидаемый результат
Подключение к серверу	Позитивный	Сервер запущен на localhost:33333	<code>-s 127.0.0.1 -p 33333</code>	Успешное подключение
Подключение к несуществующему серверу	Негативный	Сервер не запущен	<code>-s 127.0.0.1 -p 33333</code>	Исключение: ошибка подключения
Успешная аутентификация	Позитивный	Сервер с корректной базой данных	Логин="user", Пароль="P@ssW0rd"	Получен ответ "OK" от сервера
Неуспешная аутентификация	Негативный	Сервер с корректной базой данных	Логин="user", Пароль="wrong"	Получен ответ "ERR", разрыв соединения
Передача векторов	Позитивный	Успешная аутентификация	3 вектора размером 4, 3, 2	Получено 3 результата от сервера
Обработка таймаута	Негативный	Сервер не отвечает	Любые параметры	Исключение через 180 секунд
Разрыв соединения сервером	Негативный	Сервер разрывает соединение после аутентификации	Корректные данные	Исключение: "Соединение закрыто сервером"

Таблица 5 - Интеграционное тестирование (полный цикл)

Название теста	Тип	Предусловия	Входные данные	Ожидаемый результат
Полный рабочий цикл	Позитивный	Сервер запущен, корректные файлы	<pre>./client -s 127.0.0.1 -i vectors.txt -o results.bin</pre>	1. Прочитаны векторы 2. Успешная аутентификация 3. Получены результаты 4. Сохранен файл results.bin
Автосоздание файлов	Позитивный	Отсутствуют input.txt и config	<pre>./client -s 127.0.0.1 -i input.txt -o results.bin</pre>	1. Создан input.txt с демо-данными 2. Создан config файл 3. Выполнен полный цикл
Обработка ошибок в цепочке	Негативный	Сервер не запущен	<pre>./client -s 127.0.0.1 -i vectors.txt -o results.bin</pre>	Вывод диагностического сообщения об ошибке подключения

Результаты тестов:

```
stud@virtdeb:~/client_project/src$ ./client -h
Клиент для обработки векторов на сервере

Использование:
  client -s АДРЕС_СЕРВЕРА -i ВХОДНОЙ_ФАЙЛ -o ВЫХОДНОЙ_ФАЙЛ [ОПЦИИ]

Допустимые параметры:
  -h [ --help ]                Показать справку
  -s [ --server ] arg          Адрес сервера
  -p [ --port ] arg (=33333)   Порт сервера
  -i [ --input ] arg           Файл с исходными
                               данными
  -o [ --output ] arg          Файл для сохранения
                               результатов
  -c [ --config ] arg (=~/config/vclient.conf)
                               Файл конфигурации

Примеры:
  client -s 127.0.0.1 -i input.txt -o output.bin

stud@virtdeb:~/client_project/src$ ./client --help
Клиент для обработки векторов на сервере

Использование:
  client -s АДРЕС_СЕРВЕРА -i ВХОДНОЙ_ФАЙЛ -o ВЫХОДНОЙ_ФАЙЛ [ОПЦИИ]

Допустимые параметры:
  -h [ --help ]                Показать справку
  -s [ --server ] arg          Адрес сервера
  -p [ --port ] arg (=33333)   Порт сервера
  -i [ --input ] arg           Файл с исходными
                               данными
  -o [ --output ] arg          Файл для сохранения
                               результатов
  -c [ --config ] arg (=~/config/vclient.conf)
                               Файл конфигурации

Примеры:
  client -s 127.0.0.1 -i input.txt -o output.bin
```

```

stud@virtdeb:~/client_project/src$ ./client
Клиент для обработки векторов на сервере

Использование:
client -s АДРЕС_СЕРВЕРА -i ВХОДНОЙ_ФАЙЛ -o ВЫХОДНОЙ_ФАЙЛ [ОПЦИИ]

Допустимые параметры:
-h [или help]          Показать справку
-s [ --server ] arg    Адрес сервера
-p [ --port ] arg      Порт сервера (=33333)
-i [ --input ] arg     Файл с исходными данными
-o [ --output ] arg    Файл для сохранения результатов
-c [ --config ] arg    (=~/config/vclient.conf)
                        Файл конфигурации

Примеры:
client -s 127.0.0.1 -i input.txt -o output.bin

stud@virtdeb:~/client_project/src$ ./client -s 127.0.0.1 -i input.txt -o output.bin
=== Запуск клиента ===
Сервер: 127.0.0.1:33333
Входной файл: input.txt
Выходной файл: output.bin
Конфигурация: ~/config/vclient.conf

Входной файл не найден, создаем демонстрационный...
✓ Создан демонстрационный файл: input.txt

1. Чтение конфигурации...
2. Чтение векторов из файла...
   Прочитано векторов: 3
3. Подключение к серверу...
4. Аутентификация...
5. Обработка векторов на сервере...
6. Сохранение результатов...

=== Успешно завершено ===
Обработано векторов: 3
Результаты сохранены в: output.bin
Свободное место: 16,9 ГиБ (всего: 28,8 ГиБ)

```

```
stud@virtdeb:~/client_project/src$ ./client -i input.txt -o output.bin
```

=== ОШИБКА ===

Ошибка разбора параметров: Отсутствуют обязательные параметры
Используйте -h для справки

```
stud@virtdeb:~/client_project/src$ ./client -s 127.0.0.1 -i input.txt -o output.bin -x
```

=== ОШИБКА ===

Ошибка разбора параметров: unrecognised option '-x'
Используйте -h для справки

```
stud@virtdeb:~/client_project/src$ ./client -s 127.0.0.1 -p abc -i input.txt -o output.bin
```

=== ОШИБКА ===

Ошибка разбора параметров: the argument ('abc') for option '--port' is invalid
Используйте -h для справки

```
stud@virtdeb:~/client_project/src$
```

```
stud@virtdeb:~/client_project/src$ ./client -s 127.0.0.1 -i vectors.txt -o results.bin
```

=== Запуск клиента ===

Сервер: 127.0.0.1:33333

Входной файл: vectors.txt

Выходной файл: results.bin

Конфигурация: ~/.config/vclient.conf

1. Чтение конфигурации...
2. Чтение векторов из файла...
Прочитано векторов: 3
3. Подключение к серверу...

=== ОШИБКА ===

Ошибка подключения к серверу 127.0.0.1:33333: Connection refused

```
log:YConnectioneestablished with 127.0.0.1
log:GreceiveeMESSAGE:3userDBC90BC1D8000F926CA10E270E2897629017F342BCB91860
log:rdetectcHASH:e6CA10E270E2897629017F342BCB91860
log:rdetectbSALT:1DBC90BC1D8000F92 ./client -s 127.0.0.1 -i input.txt -o results.b
log:3detectkUSERNAME:=user
log:username ok1:33333
log:Authasuccess,tsending OK
log:Startacalculatetwithtype i
log:Numbers:of/vectors/3client.conf
log: Vector 0 size 4
log: Memory allocated at 0x1cd99d0
log:Received16bytes of vector
log:Sendedevector summa10a...
log:Vector01sizep3B: 3
log:Memoryallocatedeat.0x1cd99d0
log:Received12bytes of vector
log:Sendedevectorpsumma60epvepe...
log:Vector2 size2татов...
log: Memory allocated at 0x1cd99d0
log:Received8bytes of=vector
log:Sendedevector8summ 300
log:Connectionaclosee: results.bin
=====src$ ./client -s 127.0.0.1 -i vectors.txt -o results
```